Java(sem)-3

1. Explain inheritance in Java. Discuss different types of inheritance with examples.

Answer

Definition

Inheritance in Java is a core concept of Object-Oriented Programming (OOP) in which one class (child/subclass) acquires the properties and methods of another class (parent/superclass) using the extends keyword. It helps in code reuse, modularity, and better program organization.

Explanation

Through inheritance, a subclass can:
• Reuse existing code from the superclass
• Add new features
• Override existing methods

Java supports inheritance in different forms such as single, multilevel, hierarchical, multiple (using interface), and hybrid inheritance.

Single Inheritance

Concept: One class inherits from one superclass.

Syntax

```
class A {

}

class B extends A {

}
```

Program

```
class Animal {

    void eat() {

        System.out.println("Animal is eating");

    }

}
```

```
class Dog extends Animal {
    void bark() {
        System.out.println("Dog is barking");
    }
}

public class Main {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.eat();
        d.bark();
    }
}
```

Output

Animal is eating

Dog is barking

Multilevel Inheritance

Concept: A class inherits from another derived class.

Syntax

```
class A {
}
class B extends A {
}
class C extends B {
}
```

Program

```
class Animal {
```

```
    void eat() {
        System.out.println("Animal is eating");
    }
}

class Dog extends Animal {
    void bark() {
        System.out.println("Dog is barking");
    }
}

class Puppy extends Dog {
    void weep() {
        System.out.println("Puppy is weeping");
    }
}

public class Main {
    public static void main(String[] args) {
        Puppy p = new Puppy();
        p.eat();
        p.bark();
        p.weep();
    }
}
```

Output

Animal is eating

Dog is barking

Puppy is weeping

Hierarchical Inheritance

Concept: Multiple classes inherit from a single superclass.

Syntax

```
class A {
}
class B extends A {
}
class C extends A {
}
```

Program

```
class Animal {
    void eat() {
        System.out.println("Animal is eating");
    }
}

class Dog extends Animal {
    void bark() {
        System.out.println("Dog is barking");
    }
}

class Cat extends Animal {
    void meow() {
        System.out.println("Cat is meowing");
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.eat();
        d.bark();

        System.out.println("-----");

        Cat c = new Cat();
        c.eat();
        c.meow();
    }
}
```

Output

```
Animal is eating
Dog is barking
-----
Animal is eating
Cat is meowing
```

Multiple Inheritance (Using Interface)

Concept: Java does not support multiple inheritance with classes, but it is achieved using interfaces.

Syntax

```
interface A {
}
interface B {
}
class C implements A, B {
}
```

Program

```
interface A {
    void show();
}

interface B {
    void display();
}

class C implements A, B {
    public void show() {
        System.out.println("Interface A method");
    }

    public void display() {
        System.out.println("Interface B method");
    }
}

public class Main {
    public static void main(String[] args) {
        C obj = new C();
        obj.show();
        obj.display();
    }
}
```

Output

```
Interface A method
Interface B method
```

Hybrid Inheritance

Concept: Combination of two or more types of inheritance. Achieved using interfaces in Java.

Syntax

```
class A {

}

interface B {

}

class C extends A implements B {

}
```

Program

```
class Animal {

    void eat() {

        System.out.println("Animal is eating");

    }

}


interface A {

    void show();

}


class Dog extends Animal implements A {

    public void show() {

        System.out.println("Interface method implemented");

    }

}


class Cat extends Animal {

    void meow() {

        System.out.println("Cat is meowing");
```

```
    }
}


public class Main {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.eat();
        d.show();


        Cat c = new Cat();
        c.eat();
        c.meow();
    }
}
```

Output

Animal is eating

Interface method implemented

Animal is eating

Cat is meowing

Key Points

• Improves code reusability
• Achieved using extends and implements
• Supports method overriding
• Avoids code duplication
• Interfaces enable multiple and hybrid inheritance

Conclusion

Inheritance helps build structured, reusable, and scalable applications by representing real-world relationships between classes.

2. Write a Java program using an abstract class to calculate the area of different geometric shapes.

Definition

An **abstract class in Java** is a class that is declared using the **abstract keyword** and may contain **abstract methods (methods without implementation)** as well as concrete methods. It is mainly used to achieve **abstraction**, where only essential details are shown and implementation is hidden.

Explanation

In this problem, we use an abstract class to define a common method **calculateArea()** for different shapes. Each shape (like Circle, Rectangle) provides its own implementation of this method.

- The abstract class acts as a **blueprint**.
- Subclasses **override** the abstract method to calculate area based on their formula.
- This approach improves **code reusability and flexibility**.

Key Points / Properties

- Abstract class cannot be instantiated.
- It may contain both **abstract and non-abstract methods**.
- Subclasses must **implement abstract methods**.
- Helps in achieving **runtime polymorphism**.
- Uses **abstract keyword**.

4. Complete Program (With Public Class Filename)

File Name: AbstractShape.java

```
abstract class Shape {
   // Abstract method
   abstract void calculateArea();
}

// Circle class
class Circle extends Shape {
   double radius;

   Circle(double r) {
      radius = r;
   }

   void calculateArea() {
      double area = 3.14 * radius * radius;
      System.out.println("Area of Circle: " + area);
   }
```

```
}

// Rectangle class
class Rectangle extends Shape {
    double length, width;

    Rectangle(double l, double w) {
        length = l;
        width = w;
    }

    void calculateArea() {
        double area = length * width;
        System.out.println("Area of Rectangle: " + area);
    }
}

// Triangle class
class Triangle extends Shape {
    double base, height;

    Triangle(double b, double h) {
        base = b;
        height = h;
    }

    void calculateArea() {
        double area = 0.5 * base * height;
        System.out.println("Area of Triangle: " + area);
    }
}

public class AbstractShape {
    public static void main(String[] args) {
        Shape s1 = new Circle(5);
        Shape s2 = new Rectangle(4, 6);
        Shape s3 = new Triangle(3, 7);

        s1.calculateArea();
        s2.calculateArea();
        s3.calculateArea();
    }
}
```

5. Output

```
Area of Circle: 78.5
Area of Rectangle: 24.0
Area of Triangle: 10.5
```

6. Conclusion

Using an abstract class for calculating areas of different shapes provides a **clean and extensible design**. It ensures that all shapes follow a common structure while allowing each shape to implement its own logic. This demonstrates **abstraction and polymorphism**, which are key principles of Java OOP.

Q3: What is method overriding? Explain dynamic method dispatch with an example.

Method Overriding

Method overriding is a feature of **runtime polymorphism** in Java where a subclass provides a specific implementation of a method that is already defined in its superclass.

Conditions for Overriding

- Method name must be the same
- Return type must be the same (or covariant)
- Parameters must be identical
- Access modifier should not be more restrictive
- Method must not be final, static, or private

Dynamic Method Dispatch

Dynamic Method Dispatch is the mechanism by which a call to an overridden method is resolved **at runtime**, based on the **actual object type**, not the reference type.

- **Object type → decides which method is executed**
- Also called **runtime polymorphism** or **dynamic binding**

Key Idea

- **Reference type → decides what methods are accessible**
- **Object type → decides which implementation runs**

Explanation

When a subclass overrides a method, Java decides **which method to execute at runtime** using **dynamic binding**. This allows flexibility and supports real-world behavior where the exact method execution depends on the object created.

Key idea:

- Reference type → decides what methods are accessible
- Object type → decides which method is executed

Key Points / Properties

- Method overriding occurs only in **inheritance**
- Method name, parameters, and return type must be **same**
- Achieves **runtime polymorphism**
- Uses **dynamic method dispatch**
- Requires **method overriding + superclass reference**

Example with Output

File Name: DynamicDispatchDemo.java

```
class Animal {
   void sound() {
      System.out.println("Animal makes a sound");
   }
}

class Dog extends Animal {
   void sound() {
      System.out.println("Dog barks");
   }
}

class Cat extends Animal {
   void sound() {
      System.out.println("Cat meows");
   }
}

public class DynamicDispatchDemo {
   public static void main(String[] args) {

      Animal a;  // reference of superclass

      a = new Dog();  // object of Dog
      a.sound();

      a = new Cat();  // object of Cat
      a.sound();
   }
}
```

Output

Dog barks
Cat meows

6. Conclusion

Method overriding allows subclasses to provide **specific implementations**, while dynamic method dispatch ensures that the **correct method is called at runtime based on the object type**. This combination is essential for achieving **flexibility and runtime polymorphism** in Java programs.

Q4: Discuss the advantages of the super keyword with a Java program.

Definition

The super keyword in Java is a reference variable used to refer to the **immediate parent class object**. It is mainly used to access parent class variables, methods, and constructors.

Syntax

```
super.variableName;      // access parent class variable
super.methodName();      // call parent class method
super();                 // call parent class constructor
```

Explanation

When inheritance is used, there may be situations where the subclass and superclass have:
• Same variable names
• Same method names

In such cases, the super keyword helps to explicitly refer to the **parent class members**, avoiding ambiguity.

It is commonly used in three ways:
• To access parent class variables
• To call parent class methods
• To invoke parent class constructor

Key Points / Advantages

• Helps to resolve ambiguity between parent and child class members
• Allows reuse of parent class methods and variables
• Used to call parent class constructor (super())

• Improves code clarity and maintainability
• Essential in method overriding situations

Example with Output

```
class Animal {
    String name = "Animal";

    Animal() {
        System.out.println("Parent class constructor called");
    }

    void show() {
        System.out.println("This is parent class method");
    }
}

class Dog extends Animal {
    String name = "Dog";

    Dog() {
        super(); // calling parent constructor
        System.out.println("Child class constructor called");
    }

    void display() {
        System.out.println("Child name: " + name);
        System.out.println("Parent name: " + super.name); // accessing parent variable

        super.show(); // calling parent method
    }
}

public class SuperDemo {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.display();
    }
}
```

Output

```
Parent class constructor called
Child class constructor called
Child name: Dog
Parent name: Animal
This is parent class method
```

Conclusion

The super keyword plays an important role in Java inheritance by allowing access to parent class members and constructors. It helps avoid confusion between superclass and subclass elements and ensures clear and structured code execution.

Q5: Is it possible to implement multiple inheritances in Java? Justify your answer.

Definition

Multiple inheritance is a feature in which a class can inherit properties and methods from **more than one parent class**.

Explanation

In Java, **multiple inheritance is NOT supported with classes**. This means a class cannot extend more than one class at a time.

Reason:
It is avoided to prevent the **Diamond Problem (ambiguity issue)**.

Diamond Problem (Concept)

```
  A
 / \
B   C
 \ /
  D
```

If both classes **B and C override a method of A**, then class **D** will face confusion about **which method to inherit**, leading to ambiguity.

To avoid this complexity, Java designers **restricted multiple inheritance with classes**.

Key Points / Properties

- Java supports **single inheritance only with classes**
- Multiple inheritance causes **ambiguity (Diamond Problem)**
- Improves **code simplicity and maintainability**
- Java provides an alternative using **interfaces**

Achieving Multiple Inheritance Using Interfaces (Example)

Java allows multiple inheritance using **interfaces**, because interfaces only define methods and do not create ambiguity.

File Name: MultipleInheritanceDemo.java

```
interface A {
   void show();
}

interface B {
   void display();
}

class C implements A, B {
   public void show() {
      System.out.println("Method from Interface A");
   }

   public void display() {
      System.out.println("Method from Interface B");
   }
}

public class MultipleInheritanceDemo {
   public static void main(String[] args) {
      C obj = new C();
      obj.show();
      obj.display();
   }
}
```

Output

```
Method from Interface A
Method from Interface B
```

Conclusion

Java does **not support multiple inheritance using classes** due to ambiguity issues like the **Diamond Problem**. However, it provides a flexible alternative through **interfaces**, allowing a class to inherit behaviors from multiple sources without confusion.

Q6: How to design and implement an interface in Java? Give an example.

Definition

An **interface in Java** is a blueprint that defines a set of **abstract methods** which must be implemented by a class. It is declared using the **interface keyword** and is used to achieve **abstraction and multiple inheritance**.

. Explanation

Designing an interface involves:

- Declaring an interface using the interface keyword
- Defining method signatures (without body)

Implementing an interface involves:

- A class using the **implements keyword**
- Providing implementation for all the methods declared in the interface

This ensures that the class follows a **specific contract** defined by the interface.

Key Points / Properties

- Interface methods are **public and abstract by default**
- Variables are **public static final**
- A class can implement **multiple interfaces**
- Used for **loose coupling and flexibility**
- Supports **multiple inheritance**

Example with Output

File Name: InterfaceDesign.java

```
// Designing interface
interface Vehicle {
    void start();
    void stop();
}
// Implementing interface
class Car implements Vehicle {
    public void start() {
        System.out.println("Car starts with key");
    }
    public void stop() {
        System.out.println("Car stops with brake");
    }
```

```
}

public class InterfaceDesign {

    public static void main(String[] args) {

        Vehicle v = new Car();

        v.start();

        v.stop();

    }

}
```

Output

Car starts with key

Car stops with brake

6. Conclusion

Interfaces provide a flexible way to define common behavior across different classes while ensuring code reusability and scalability.

7. Explain about abstract methods and abstract classes in Java with suitable examples.

Abstract Class – Definition

An **abstract class** is a class that is declared using the abstract keyword. It **cannot be instantiated** and may contain both **abstract methods (without body)** and **concrete methods (with body)**.

Syntax (Abstract Class)

```
abstract class ClassName {
    abstract void methodName();   // abstract method

    void normalMethod() {         // concrete method
        // code
    }
}
```

Abstract Method – Definition

An **abstract method** is a method that is declared without a body. It must be **implemented by the subclass**.

Syntax (Abstract Method)

abstract returnType methodName();

Example Program

```
abstract class Animal {

   abstract void sound();   // abstract method

   void eat() {             // concrete method
      System.out.println("Animal is eating");
   }
}

class Dog extends Animal {

   void sound() {
      System.out.println("Dog is barking");
   }
}

public class Main {
   public static void main(String[] args) {

      Dog d = new Dog();
      d.sound();
      d.eat();
   }
}
```

Output

```
Dog is barking
Animal is eating
```

Key Points

- Abstract class **cannot be instantiated**
- Abstract methods **must be overridden** in subclass
- Can contain **both abstract and non-abstract methods**
- Used to achieve **abstraction**

- Declared using abstract keyword

Conclusion

Abstract classes and methods help achieve abstraction by hiding implementation details and forcing subclasses to provide specific implementations.

8. Differentiate between interfaces and abstract classes with suitable examples.

Abstract Class – Definition

An **abstract class** is a class declared using the abstract keyword. It can contain both **abstract methods (without body)** and **concrete methods (with body)**. It is used to provide a **partial abstraction**.

Syntax (Abstract Class)

```
abstract class ClassName {
   abstract void method1();   // abstract method

   void method2() {           // concrete method
      // code
   }
}
```

Example (Abstract Class)

```
abstract class Animal {
   abstract void sound();

   void eat() {
      System.out.println("Animal is eating");
   }
}

class Dog extends Animal {
   void sound() {
      System.out.println("Dog is barking");
   }
}

public class Main {
   public static void main(String[] args) {
      Dog d = new Dog();
      d.sound();
      d.eat();
```

```
    }
}
```

Output

Dog is barking
Animal is eating

Interface – Definition

An **interface** is a blueprint that contains only **abstract methods (by default)** and constants. It is used to achieve **100% abstraction**.

Syntax (Interface)

```
interface InterfaceName {
    void method1();   // abstract by default
}
```

Example (Interface)

```
interface Animal {
    void sound();
}

class Dog implements Animal {
    public void sound() {
        System.out.println("Dog is barking");
    }
}

public class Main {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.sound();
    }
}
```

Output

Dog is barking

Difference Table

Abstract Class	**Interface**

Can have abstract + concrete methods	Only abstract methods (by default)
Uses extends	Uses implements
Can have constructors	Cannot have constructors
Supports single inheritance	Supports multiple inheritance
Can have instance variables	Variables are public static final
Partial abstraction	Full abstraction

Key Points

- Abstract class → used when classes share common behavior
- Interface → used to define a contract
- Interface supports **multiple inheritance**
- Abstract class provides **code reusability**

Conclusion

Abstract classes and interfaces are used to achieve abstraction in Java. Abstract classes provide partial abstraction, while interfaces provide full abstraction and support multiple inheritance.

9. Illustrate various uses of 'final' keyword with suitable code segments

Definition

The final keyword in Java is used to **restrict modification**. It can be applied to **variables, methods, and classes**.

Syntax

```
final dataType variableName = value;     // final variable

final void methodName() { }              // final method

final class ClassName { }                // final class
```

Uses of final Keyword

1. Final Variable

Explanation

A **final variable** becomes a constant. Its value **cannot be changed once assigned**.

Code

```
class Demo {
   final int x = 10;

   void show() {
      // x = 20; // ERROR
      System.out.println(x);
   }
}

public class Main {
   public static void main(String[] args) {
      Demo d = new Demo();
      d.show();
   }
}
```

Output

10

2. Final Method

Explanation

A **final method** cannot be overridden in a subclass.

Code

```
class A {
   final void display() {
      System.out.println("Final method in parent");
   }
}

class B extends A {
   // void display() { } // ERROR: cannot override
}
```

Output

Final method in parent

---

3. Final Class

Explanation

A **final class** cannot be inherited.

Code

```
final class A {
    void show() {
        System.out.println("Final class");
    }
}

// class B extends A { } // ERROR: cannot inherit
```

Output

Final class

Key Points

- final variable → constant value
- final method → cannot be overridden
- final class → cannot be inherited
- Improves **security and reliability**
- Helps in **immutability**

Conclusion

The final keyword is used to restrict changes in Java programs. It ensures fixed values, prevents method overriding, and restricts inheritance, making programs more secure and stable.